

# INFO288: Buscador tipo Google Search V16

Martin Alvarado, Eduardo Barrera, Isaias Cabrera, Osvaldo Casas-Cordero, Andrés Mardones

## Resumen

Este informe presenta el diseño arquitectónico de un motor de búsqueda distribuido de alto rendimiento orientado a mitigar las ineficiencias de indexación y latencia en repositorios de literatura científica académica. La solución reemplaza los enfoques secuenciales tradicionales mediante la implementación de una arquitectura modular de cuatro capas acoplada de forma nativa a un clúster de Elasticsearch v8.12. El sistema incorpora un pipeline asíncrono de procesamiento de lenguaje natural (NLP) encargado de normalizar, tokenizar y estructurar índices invertidos dinámicos desde repositorios centralizados en Google Drive mediante Cuentas de Servicio institucionales. Para absorber altas cargas concurrentes de hasta 100 consultas por segundo, se integra una capa de caché multinivel basada en Redis v7.4.0 que reduce los tiempos de respuesta a milisegundos en búsquedas recurrentes. La infraestructura se distribuye estratégicamente a través de una topología de cinco nodos de cómputo heterogéneos que mitigan fallas de hardware mediante estrategias de fragmentación dinámica (*sharding*) y replicación cruzada. El diseño garantiza un nivel de disponibilidad del 99.5 % y una consistencia eventual máxima de 5 minutos, proveyendo un ecosistema tecnológico escalable, seguro bajo protocolos mTLS/HTTPS y optimizado para el consorcio de investigación institucional.

## I. INTRODUCCIÓN

EN el presente informe se describe el diseño arquitectónico de un sistema distribuido enfocado en resolver la indexación y localización ineficiente de literatura científica y académica dentro de un consorcio de investigación institucional. Para eso se presenta en primer lugar los detalles de la problemática, así como las hipotéticas especificaciones que una corporación podría entregar para responder preguntas orientadas a complementar la información con respecto al contexto de la empresa y entregar un producto más específico.

Luego, se detalla la solución propuesta, que está basada en un sistema distribuido. Un sistema distribuido es un conjunto de componentes independientes que trabajan de forma coordinada para actuar como un único sistema coherente frente al usuario. Este tipo de sistemas presentan múltiples ventajas relacionadas con el rendimiento, escalabilidad o el manejo de fallas y errores. Es por eso que el objetivo del proyecto es aprovechar estos beneficios con tal de desarrollar un buscador capaz de facilitar la localización de información entre una vasta cantidad de documentos. Así, el sistema será capaz de soportar un gran volumen de consultas y, además, podrá escalar de forma adecuada ante un crecimiento exponencial de documentos.

## II. PROBLEMA DETECTADO (U OPORTUNIDAD DE INNOVACIÓN)

### II-A. Descripción

En la actualidad, el volumen de literatura científica, tesis de grado y publicaciones académicas generado por las distintas facultades crece de forma exponencial. El problema reside en la latencia y la ineficiencia de los métodos de búsqueda tradicionales basados en escaneo secuencial de archivos o consultas de texto parcial en bases de datos relacionales, los cuales no escalan ante miles de documentos enriquecidos de muchas páginas.

Limitaciones detectadas:

- **Velocidad de respuesta:** A medida que aumenta el volumen de artículos y tesis indexadas, el tiempo de búsqueda semántica dentro del cuerpo del texto se vuelve demasiado extenso para los investigadores y estudiantes.
- **Dispersión de fuentes:** La dificultad de consolidar y clasificar documentos PDF y DOCX provenientes de diferentes repositorios o facultades en una interfaz única y amigable.
- **Costo computacional:** Repetir cálculos y extracciones complejas de texto para búsquedas de términos frecuentes agota los recursos del servidor académico sin necesidad.

La oportunidad radica en implementar una arquitectura de Sistema Distribuido que no solo centralice el acceso a la literatura, sino que la procese de manera inteligente mediante etapas de preprocesamiento de texto para ofrecer resultados instantáneos. La innovación se centra en tres pilares:

1. **Indexación mediante Índice Invertido:** En lugar de buscar una palabra dentro de cada paper o PDF, se crea un diccionario donde cada palabra (token) apunta a una lista de documentos que la contienen. Esto transforma una búsqueda de complejidad lineal en una búsqueda de acceso casi instantáneo.
2. **Estrategia de Caché Multinivel:** Implementar una capa de memoria volátil que almacene los resultados de las consultas académicas más frecuentes. Esto permite que, para términos populares o temas de contingencia científica, el sistema responda en milisegundos sin consultar el índice en disco.
3. **Escalabilidad Horizontal:** Al ser un sistema distribuido, la carga del preprocesamiento, la indexación y la búsqueda se reparte entre varios nodos, permitiendo que el buscador crezca orgánicamente según aumente el patrimonio bibliográfico de la institución.

## II-B. Preguntas y Supuestos

Los supuestos de este sistema se presentan mediante preguntas a los clientes, junto con sus respectivas respuestas, configurando los requerimientos del sistema:

- ¿**Qué tipo de documentos van a ser buscados?** R. Documentos en formatos complejos como **PDF** (artículos y revistas terminadas) y **DOCX** (tesis y borradores académicos), además de extensiones más comunes como **MD** y **TXT**.
- ¿**Cuál es el volumen de consultas esperado?** R. Se esperan al menos 100 consultas por segundo en períodos de alta demanda académica.
- ¿**Cuál es el origen de los documentos? ¿De dónde se ingresan al sistema?** R. El origen de los documentos será un repositorio institucional centralizado en Google Drive, donde las facultades cargan sus publicaciones.
- ¿**Cuántos servidores se tienen y cuáles son sus características?** R. Se cuentan con 5 computadores especificados en el apartado *IV Modelo físico tabla II*.
- ¿**Con cuántos documentos se cuenta al inicio y cuál es la tasa de crecimiento esperada?** R. Inicialmente manejamos un volumen de aproximadamente **10,000 documentos complejos**. Esperamos crecer un 30 % anual a medida que se digitalicen e integren archivos históricos.
- Con respecto a actualizaciones de documentos, ¿se deben reflejar los cambios en tiempo real o es aceptable un retraso?** R. Es aceptable un máximo de 5 minutos de retraso para reflejar los cambios (consistencia eventual).
- ¿**Cuál es el nivel de disponibilidad requerido?** R. Si bien no es un sistema crítico médico o financiero, queremos una alta disponibilidad (99.5 %) para facilitar el trabajo continuo de los investigadores de la red.
- ¿**Necesitan herramientas de monitoreo y alertas sobre el estado del sistema?** R. Por el momento no consideramos necesario implementar herramientas de monitoreo externas para simplificar la infraestructura inicial y mantener los costos bajos.
- ¿**Qué aspectos de accesibilidad son requeridos para la interfaz?** R. Debe incluir obligatoriamente el modo nocturno y una opción para activar colores de alto contraste para usuarios con dificultades visuales.
- ¿**Qué aspectos debe tener en cuenta la interfaz de usuario?** R. Necesitamos una interfaz web simple, rápida y limpia, optimizada para búsquedas por palabras clave, similar al buscador de Google o SciELO.

## III. SOLUCIÓN PROPUESTA Y DISEÑO ARQUITECTÓNICO

### III-A. Estructura del Modelo y Capas del Sistema

El sistema se organiza en cuatro capas horizontales con responsabilidades estrictamente separadas, siguiendo el principio de que ninguna capa puede saltarse a otra que no sea la adyacente. Cada componente desarrollado internamente se empaquetará de forma aislada utilizando tecnología de **containers (Docker)** bajo la imagen oficial `python:3.11-slim`, garantizando portabilidad y aislamiento, a excepción de los motores de bases de datos que irán directo sobre el servidor físico según requerimientos de infraestructura.

**Capa de Presentación (Frontend):** Es el punto de contacto con los usuarios. Desde un navegador web, estudiantes, académicos y funcionarios emiten sus consultas. Se compone por la **Interfaz Gráfica**, desarrollada en Astro v6.13 (JavaScript) y montada sobre un contenedor Docker. Su único rol es recibir la solicitud, iniciar el proceso de autenticación contra el sistema CAS institucional, y mostrar los resultados ordenados. Expone públicamente el puerto 3000.

**Capa de Coordinación y Seguridad (API Gateway):** Es el núcleo inteligente del sistema. Aquí vive el **Query Router**, desarrollado en Python 3.11 con el framework **FastAPI v0.135** (contenedorizado, expone el puerto público 8000), el filtro ACL y la **Caché** de alto rendimiento basada en **Redis v7.4.0** (instalada de forma nativa *bare-metal* en el servidor para evitar sobrecargas de red). El Query Router recibe la consulta, la tokeniza y decide si la respuesta ya existe en caché o debe generarse consultando los índices.

**Capa de Procesamiento Distribuido (Index Nodes y Ranker):** Contiene el componente **Merger + Ranker** (integrado en Elasticsearch). Cuando ocurre un *Cache Miss*, este componente se comunica con el Query Router mediante **HTTP** sobre TCP (puerto privado 9200) para otorgar máxima agilidad interna. Reúne las listas de documentos, las intersecta y aplica un algoritmo de scoring compuesto.

**Capa de Almacenamiento Persistente (Bases de Datos y Repositorio):** Contiene los **Index Nodes** montados sobre **Elasticsearch v8.12**, el cual se instala de forma nativa directo sobre el servidor físico (bare-metal). Se seleccionó Elasticsearch en lugar de soluciones columnares como Cassandra debido a que está construido nativamente sobre Apache Lucene, proveyendo de forma directa la arquitectura de índice invertido y *posting lists* ideal para búsquedas semánticas empresariales. Los metadatos estructurados (autor, año, facultad) se almacenan de forma denormalizada en el mismo índice. El origen de los archivos es el **Repositorio de Documentos** basado en una cuenta institucional de Google Drive.

**Componente de Ingesta (Parser + Indexer):** Servicio asíncrono en Python 3.11 (contenedorizado en Docker) encargado de escanear Google Drive mediante la librería oficial `google-api-python-client` y `google-auth-oauthlib`. Para evadir las limitaciones de expiración de tokens de usuarios humanos, la conexión implementa una **Cuenta de Servicio (Service Account)** autenticada por llaves privadas JSON. Utiliza las bibliotecas `pypdf` y `python-docx` como pipeline de preprocesamiento NLP para extraer texto de archivos complejos, remover *stopwords* y aplicar *stemming* antes de indexar en Elasticsearch.

### III-B. Diagrama de Componentes

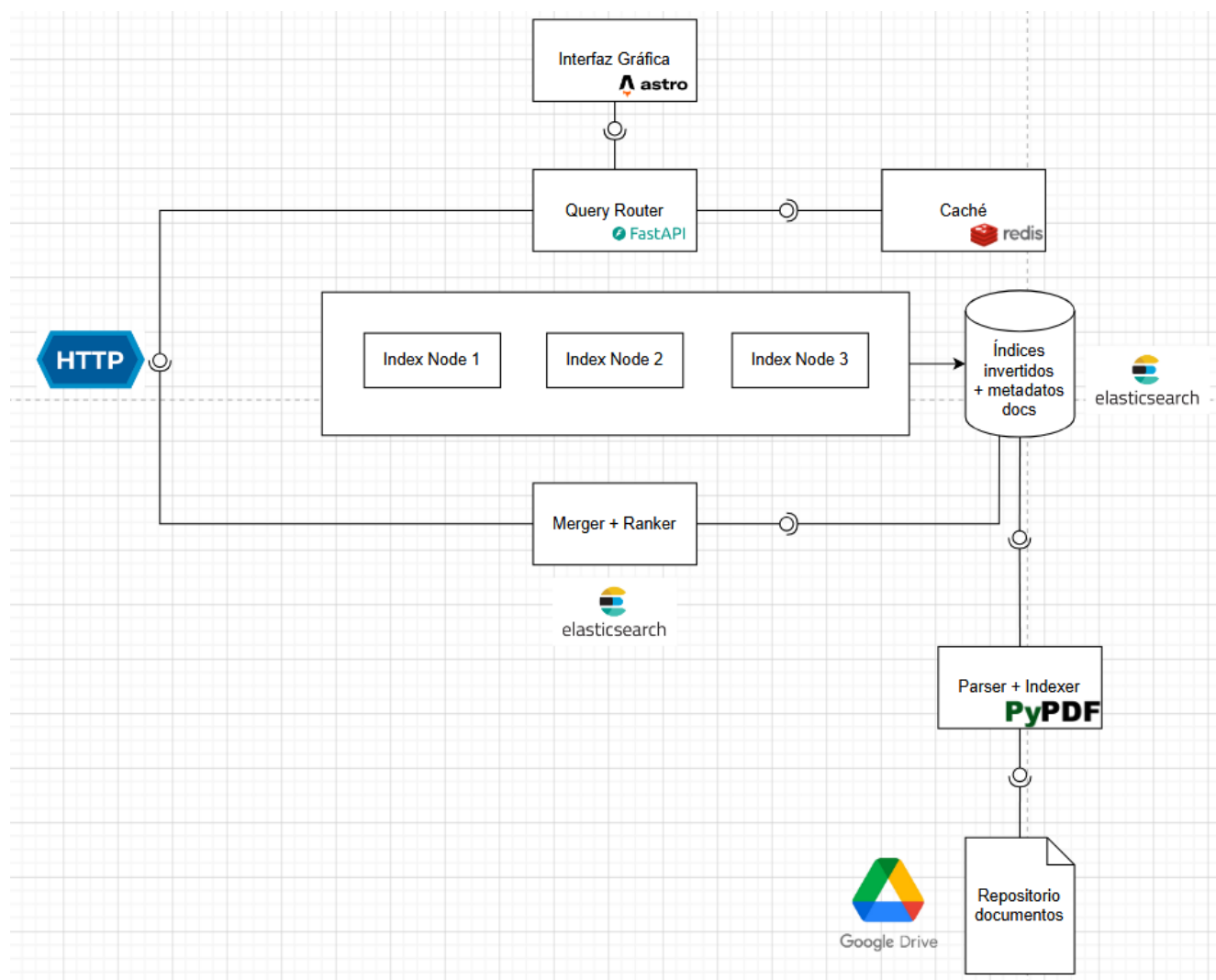


Figura 1. Diagrama de componentes del sistema de búsqueda distribuido basado en capas de servicio, contenedores Docker y almacenamiento nativo.

### III-C. Modelos Fundamentales

**III-C1. Interacción y Flujos de Trabajo:** A continuación se explica cómo se comportará el sistema con respecto a los casos de usos más importantes:

- **Caso Búsqueda (Flujo Sincrónico):** El sistema recibe la consulta del usuario en el Frontend mediante una cadena de texto. El *Query Router* procesa la solicitud (asíncrona) y procede a buscar en la caché mediante el protocolo RESP de REDIS. En el caso de haber un *caché hit*, se devuelve la lista de resultados calculada en milisegundos.  
Si existe un *caché miss*, se continúa la búsqueda hacia el *Merger + Ranker*. Este componente interroga a los *Index Nodes* de *Elasticsearch* para obtener las estadísticas léxicas de las palabras y los metadatos de los documentos. El *Merger + Ranker* computa el ranking compuesto y envía la información compacta al *Query Router*. **Antes de entregar la respuesta al usuario, el Query Router almacena inmediatamente este nuevo vector de resultados en la caché Redis con un tiempo de vida útil (TTL) definido, garantizando que búsquedas académicas idénticas subsecuentes no saturen los discos del servidor.**
- **Caso Nuevo Archivo (Flujo Asíncrono):** Cuando un nuevo artículo científico o tesis es subido al repositorio de Google Drive, el componente *Parser + Indexer* registra el evento mediante el Service

Account. El componente descarga el binario (PDF/DOCX), extrae el contenido textual mediante el pipeline de preprocesamiento NLP e indexa el documento con sus metadatos en *Elasticsearch*. Finalmente, ejecuta una invalidación selectiva en la caché *Redis* para los tokens modificados, asegurando que las nuevas búsquedas reflejen el documento en un tiempo máximo de 5 minutos.

- **Caso Eliminación de un Archivo:** Para eliminar un archivo, el sistema detecta la baja en Google Drive y procede a registrar el ID del documento en un índice rápido de exclusión lógica dentro de *Elasticsearch*. El *Merger + Ranker* consulta este índice en cada búsqueda, filtrando los resultados de inmediato para que el usuario no vea literatura obsoleta. Dado el gran volumen de datos del consorcio universitario, modificar y reconstruir las *posting lists* de los índices invertidos en tiempo real generaría una sobrecarga crítica en los servidores. Por lo tanto, el rastro físico del documento se elimina por completo de los *Index Nodes* mediante un **proceso de reindexación total estructurado a través de un Cron Job programado semanalmente a las 02:00 AM**, mitigando cualquier impacto en la latencia diaria.
- **Caso Modificación de un Archivo:** El sistema opera bajo una lógica distribuida de “eliminar y crear”. Primero se ejecuta el borrado lógico del documento agregando su ID al índice de exclusión e invalidando las llaves afectadas en la caché *Redis*. Inmediatamente después, el *Parser + Indexer* procesa el archivo modificado como si fuese un documento completamente nuevo, indexándolo en *Elasticsearch* con un identificador actualizado.

**III-C2. Manejo de Fallos y Tolerancia:** Dado que el sistema opera sobre 5 nodos con hardware distinto, la estrategia se centra en la tolerancia a fallas para asegurar la continuidad operativa del ecosistema SciELO institucional.

Tabla I  
MATRIZ DE FALLOS Y ESTRATEGIAS DE MITIGACIÓN DISTRIBUIDAS

| Tipo de Fallo            | Componente       | Estrategia de Mitigación                                                                                                                                                                                                                                                   |
|--------------------------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Caída de Nodo de Índices | Elasticsearch    | Los índices se configuran particionados en fragmentos ( <i>shards</i> ) con un factor de replicación $R = 2$ . Si el nodo primario de un fragmento cae, el nodo de réplica asume de forma transparente las lecturas.                                                       |
| Falla en memoria Caché   | Redis (Caché)    | Si el servicio de Redis no responde, el <i>Query Router</i> captura la excepción de conexión y degrada el sistema redirigiendo temporalmente las consultas directo al <i>Merger + Ranker</i> . El sistema experimentará mayor latencia, pero preservará la disponibilidad. |
| Bloqueo de Cuota o Token | API Google Drive | Si las peticiones al repositorio exceden las cuotas de Google, el <i>Parser</i> mitiga el fallo suspendiendo temporalmente la ingesta, volcando el estado en un log e implementando reintentos automáticos con respaldo exponencial de la Cuenta de Servicio.              |
| Sobrecarga de Consultas  | Backend / API    | Ante picos de concurrencia superiores a las 100 RPS, el <i>Query Router</i> aplicará políticas de control de flujo en la API de FastAPI para evitar el desborde de la memoria RAM en los nodos de cómputo más débiles.                                                     |

**III-C3. Diseño de Seguridad:** El diseño de seguridad del sistema distribuido se enfoca en proteger los canales, procesos y objetos, mitigando vulnerabilidades tanto externas como internas. A nivel perimetral, la interacción con el usuario se realiza mediante HTTPS para establecer canales seguros que aseguran la privacidad e integridad de las consultas. Además, para evitar la saturación del servidor producto de una sobrecarga de solicitudes y sostener el rendimiento esperado, el nodo de entrada aplica limitación de tasa (*Rate Limiting*), previniendo amenazas de negación de servicios (*DoS*). Como medida preventiva secundaria frente a la potencial inyección de código malicioso a los servidores, los documentos ingresan a la plataforma habiendo sido previamente saneados por un proceso externo.

Para resguardar la infraestructura interna, la comunicación entre los componentes del buscador utiliza el protocolo HTTPS respaldado por autenticación mutua (*mTLS*). Esto impide que un proceso atacante ocupe una identidad falsa para acceder a la memoria caché o los nodos de indexación. Mediante el uso de técnicas de

encriptación y secreto compartido, los servidores validan rigurosamente la identidad de cada nodo antes de procesar cualquier solicitud interna, garantizando un entorno seguro y de confianza estricta.

#### IV. MODELO FÍSICO

Tabla II  
CARACTERÍSTICAS DE LA INFRAESTRUCTURA DE CÓMPUTO

| Comp. | CPU           | GPU               | RAM   | Almacenamiento        |
|-------|---------------|-------------------|-------|-----------------------|
| 1     | Ryzen 5 4600H | GTX 1650Ti        | 8 GB  | 512 GB SSD            |
| 2     | i5-11400H     | GTX 1650          | 16 GB | 512 GB SSD            |
| 3     | i5-12500H     | Iris® Xe Graphics | 16 GB | 256,1 GB SSD          |
| 4     | i5-11400H     | GTX 1650          | 16 GB | 256 GB SSD            |
| 5     | i5-14400F     | RX 6700 XT        | 16 GB | 931GB HDD / 832GB SSD |

#### V. MODELO DE ALMACENAMIENTO Y ESTRUCTURA DE DATOS

El sistema utiliza una arquitectura de almacenamiento distribuida basada en *Elasticsearch v8.12*, aprovechando su alta eficiencia en la indexación de texto y su capacidad nativa de escalabilidad horizontal. Al estructurar el sistema en torno a un motor de búsqueda puro, los datos abandonan el esquema rígido de tablas relacionales o columnares y se organizan mediante documentos de objetos JSON y mapeos lógicos (*mappings*). Esto optimiza la latencia de las consultas al almacenar la información denormalizada, evitando operaciones de unión (*joins*) costosas en el entorno de red distribuido.

##### V-A. Estructura Interna del Índice Invertido

Para resolver la localización instantánea de términos dentro de la literatura académica, el sistema fundamenta su rendimiento en la estructura de un **Índice Invertido** gestionada a nivel de núcleo por el motor Apache Lucene.

Cuando el componente *Parser + Indexer* procesa un documento binario (PDF o DOCX) proveniente de Google Drive, el texto extraído se somete a un pipeline de análisis léxico que tokeniza las palabras, remueve las palabras vacías (*stopwords*) y reduce los términos a su raíz gramatical (*stemming*). Con este resultado, Elasticsearch construye y actualiza el índice unificado denominado *academic\_inverted\_index*, el cual se compone internamente de dos estructuras:

**Diccionario de Términos (Term Dictionary):** Una estructura ordenada que almacena todos los tokens únicos extraídos del corpus documental. Cuenta con un índice en memoria RAM (*Term Index*) que funciona como un árbol de búsqueda rápida para localizar la ubicación física de un término en el disco sin recorrerlo secuencialmente.

**Lista de Publicaciones (Posting List):** Un vector dinámico vinculado a cada palabra del diccionario que apunta exclusivamente a los identificadores (*doc\_id*) de los documentos que contienen dicho término. Cada entrada de la lista almacena la frecuencia exacta de la palabra (*Term Frequency*) y sus posiciones para dar soporte al algoritmo de ordenamiento y puntuación estadística BM25.

La Tabla III muestra el mapeo conceptual de cómo se estructuran e intersecan internamente los términos y sus vectores de documentos dentro del clúster.

Tabla III  
MAPEO CONCEPTUAL DE ESTRUCTURAS INTERNAS EN *academic\_inverted\_index*

| Diccionario (Término) | Posting List ( <i>doc_id</i> ) | Atributos Internos (Frecuencia / Posición)                     |
|-----------------------|--------------------------------|----------------------------------------------------------------|
| distribuido           | Doc_001, Doc_045               | [Doc_001: freq=3, pos=[12, 85]], [Doc_045: freq=1, pos=[4]]    |
| algoritmo             | Doc_012, Doc_045               | [Doc_012: freq=5, pos=[1]], [Doc_045: freq=2, pos=[10, 50]]    |
| scielo                | Doc_001, Doc_012               | [Doc_001: freq=1, pos=[90]], [Doc_012: freq=4, pos=[2, 8, 15]] |

### V-B. Esquema de Documentos y Metadatos

A diferencia de las bases de datos relacionales que separan los datos en múltiples tablas, Elasticsearch almacena los metadatos de los archivos de forma denormalizada dentro de un objeto estructurado embebido en el mismo documento JSON del índice principal. Esto permite que, una vez resuelta la intersección de las *Posting Lists*, el *Merger + Ranker* recupere la información del archivo de forma directa en una sola operación de lectura.

La Tabla IV detalla los tipos de datos lógicos definidos en el mapeo (*mapping*) de Elasticsearch para representar los metadatos de los documentos indexados.

Tabla IV  
ESPECIFICACIÓN DEL MAPEO DE METADATOS DEL DOCUMENTO

| Campo JSON      | Tipo de Dato | Descripción y Propósito Técnico                                                                   |
|-----------------|--------------|---------------------------------------------------------------------------------------------------|
| doc_id          | keyword      | Identificador único universal (UUID) del documento, utilizado para emparejar las búsquedas.       |
| file_name       | text         | Nombre original del archivo cargado (ej: <i>"tesis_final.pdf"</i> ). Permite búsquedas parciales. |
| download_url    | keyword      | Enlace directo para la descarga del archivo binario desde el repositorio Google Drive.            |
| file_size_bytes | long         | Tamaño total del archivo almacenado medido en bytes.                                              |
| word_count      | integer      | Cantidad total de palabras extraídas y procesadas en el documento.                                |
| indexed_at      | date         | Marca de tiempo en formato ISO-8601 que registra el ingreso del documento al sistema.             |

### V-C. Índice de Exclusión Lógica para Eliminaciones

Para gestionar las solicitudes de eliminación de manera asíncrona sin incurrir en el alto costo computacional de reescribir físicamente las *Posting Lists* en disco en tiempo real, el sistema utiliza un índice ligero complementario denominado *deleted\_papers*. Este índice actúa como una lista de exclusión activa que es consultada en memoria por el componente *Merger + Ranker* para filtrar los resultados antes de ser enviados al usuario.

Tabla V  
ESPECIFICACIÓN DEL ÍNDICE DE EXCLUSIÓN: *deleted\_papers*

| Campo JSON | Tipo de Dato | Descripción Técnica                                                                  |
|------------|--------------|--------------------------------------------------------------------------------------|
| doc_id     | keyword      | Identificador universal del documento académico que fue removido de Google Drive.    |
| deleted_at | date         | Registro de fecha y hora exacta del marcado de borrado para el control del Cron Job. |